

PROJECT REPORT

Credit Card Approval Prediction Using Machine Learning

Submitted by:

Gandham Manikanta	S210143
Dandu Aniha	R210463
Kudupudi Akhila Devi	S210385
Jahnavi Duvvada	S210163

RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Academic Year 2026 - 2027

Table of Contents

1. Introduction.....	3
1.1 Project Overview.....	3
1.2 Purpose.....	3
2. Ideation Phase.....	4
2.1 Problem Statement.....	4
2.2 Empathy Map Canvas.....	4
2.3 Brainstorming.....	5
3. Requirement Analysis.....	6
3.1 Customer Journey Map.....	6
3.2 Solution Requirement.....	6
3.3 Data Flow Diagram.....	7
3.4 Technology Stack.....	7
4. Project Design.....	8
4.1 Problem - Solution Fit.....	8
4.2 Proposed Solution.....	8
4.3 Solution Architecture.....	9
4.4 System Workflow.....	9
5. Project Planning & Scheduling.....	11
5.1 Project Planning.....	11
6. Model Building and Performance Testing.....	12
6.1 Algorithm Overview.....	12
6.2 Performance Testing.....	13
7. Results.....	14
7.1 Output Screenshots.....	14
8. Advantages & Disadvantages.....	15
9. Conclusion.....	15
10. Future Scope.....	16
11. Appendix.....	16

1. Introduction

1.1 Project Overview

The project "Credit Card Approval Prediction Using Machine Learning" is an intelligent decision-support system designed to automate the evaluation of credit card applications submitted to a bank or financial institution. Rather than relying entirely on manual verification by loan officers, the system analyzes an applicant's financial and credit-related attributes - such as income, existing loan obligations, credit history, number of recent credit inquiries, and past repayment behaviour - and predicts whether the application should be approved or rejected.

The system is built using Python as the core programming language, with the machine learning pipeline developed using NumPy, Pandas, Scikit-learn, and XGBoost. Four classification algorithms - Logistic Regression, Decision Tree, Random Forest, and XGBoost - are trained and compared on historical applicant data, and the best-performing model is persisted and served through a Flask web application. To support scalable, production-style access to the trained model, the final solution is additionally deployed on IBM Watson Machine Learning, exposing the prediction logic as a cloud-hosted API that the Flask front end consumes.

1.2 Purpose

The purpose of this project is to reduce the time, cost, and inconsistency associated with manual credit card approval decisions. Traditional approval workflows require staff to manually cross-check applicant details against internal policies, which is slow when application volumes are high and is susceptible to human error and inconsistent judgement between different reviewers. This project addresses that gap by:

- ▯ Automatically classifying incoming applications as likely to be approved or rejected based on patterns learned from historical data.
- ▯ Flagging high-risk applicants - those with poor repayment history or excessive credit inquiries - for closer manual review.
- ▯ Allowing prospective customers to check their likely eligibility before formally applying, reducing wasted effort on both sides.
- ▯ Giving the bank a consistent, auditable, and data-driven basis for approval decisions that can be monitored and improved over time.

Ultimately, the system aims to make the credit approval process faster, more consistent, and less dependent on manual effort, while still allowing human oversight for borderline or high-risk cases.

2. Ideation Phase

2.1 Problem Statement

Banks and financial institutions receive a large volume of credit card applications on a daily basis. A significant proportion of these are ultimately rejected because of factors such as high existing loan balances, low income relative to the requested credit limit, poor credit history, an unusually large number of recent credit inquiries, or a history of payment defaults. Verifying every application manually against these criteria is time-consuming and expensive, and different reviewers may reach different conclusions for similar applicant profiles, leading to inconsistent outcomes. There is a need for a system that can take an applicant's financial and credit attributes and produce a reliable, consistent approve/reject prediction, while still surfacing the key risk factors behind that prediction for human reviewers.

2.2 Empathy Map Canvas

An empathy map was used to understand the perspective of the primary users of the system - the bank's credit analysts and compliance officers - as well as the applicants themselves, across what they say, think, do, and feel during the credit evaluation process.

Says	Thinks	Does	Feels
"I have too many applications to review manually every day."	"Am I applying the approval policy consistently across every applicant?"	Manually cross-checks income, credit history, and inquiry count against internal rules	Overworked and concerned about inconsistent decisions
"Why was my application rejected? I was never told the reason."	"Is there something I can improve about my finances before reapplying?"	Applies for a credit card without knowing their approval likelihood in advance	Uncertain and occasionally frustrated by a lack of transparency

2.3 Brainstorming

Before converging on the final scope, the following features and directions were brainstormed:

- A prediction engine that classifies each application as "Approved" or "Rejected" using historical applicant data.
- A risk-scoring module that flags applicants with poor payment history or excessive credit inquiries for manual review.
- A self-service eligibility checker that lets prospective customers estimate their approval chances before formally applying.
- A comparison of multiple algorithms (Logistic Regression, Decision Tree, Random Forest, XGBoost) to identify the most reliable model for deployment.

- Cloud deployment of the trained model via IBM Watson Machine Learning, so the prediction service can be accessed as an API independent of the Flask front end.



Brainstorm & idea prioritization

Use this template in your own brainstorming sessions so your team can unleash their imagination and start shaping concepts even if you're not sitting in the same room.

🕒 10 minutes to prepare
🕒 1 hour to collaborate
👤 2-8 people recommended

➔

Before you collaborate

A little bit of preparation goes a long way with this session. Here's what you need to do to get going.

🕒 10 minutes

A **Team gathering**
Define who should participate in the session and send an invite. Share relevant information or pre-work ahead.

B **Set the goal**
Think about the problem you'll be focusing on solving in the brainstorming session.

C **Learn how to use the facilitation tools**
Use the Facilitation Superpowers to run a happy and productive session.

[Open article](#) ➔

1

Define your problem statement

What problem are you trying to solve? Frame your problem as a How Might We statement. This will be the focus of your brainstorm.

🕒 5 minutes

PROBLEM

How might we [your problem statement]?



Key rules of brainstorming

To run a smooth and productive session

😊 Stay in topic.	💡 Encourage wild ideas.
⏸️ Defer judgment.	👂 Listen to others.
🗣️ Go for volume.	👁️ If possible, be visual.

3. Requirement Analysis

3.1 Customer Journey Map

The journey map below traces a bank analyst's experience from receiving an application to acting on the system's prediction.

Stage	Analyst Action	Touchpoint	Opportunity
Application Received	Logs applicant details into the system	Application intake form / internal portal	Reduce manual data entry through digital intake
Screening	Submits applicant data to the prediction system	Flask web application	Keep the input form short and clearly labelled
Prediction	Receives an approve/reject prediction with supporting factors	Results page	Explain which factors drove the prediction
Verification	Cross-checks borderline predictions manually	Results page, internal policy documents	Highlight only borderline cases needing manual review
Decision	Approves, rejects, or escalates the application	Internal approval system	Log outcomes to continually retrain and improve the model

3.2 Solution Requirement

Functional Requirements

ID	Requirement	Description
FR-1	Data Input	User can enter applicant details - income, existing loan balance, credit history length, number of credit inquiries, and past default status - via a web form
FR-2	Approval Prediction	System returns a prediction of "Approved" or "Rejected" for the entered applicant profile
FR-3	Risk Explanation	System highlights the key factors (e.g. high inquiries, prior defaults) contributing to a rejection
FR-4	Model Comparison	System is trained and evaluated across multiple algorithms, with the best-performing one selected for deployment
FR-5	Cloud Prediction API	Trained model is deployed on IBM Watson Machine Learning and exposed as a callable prediction

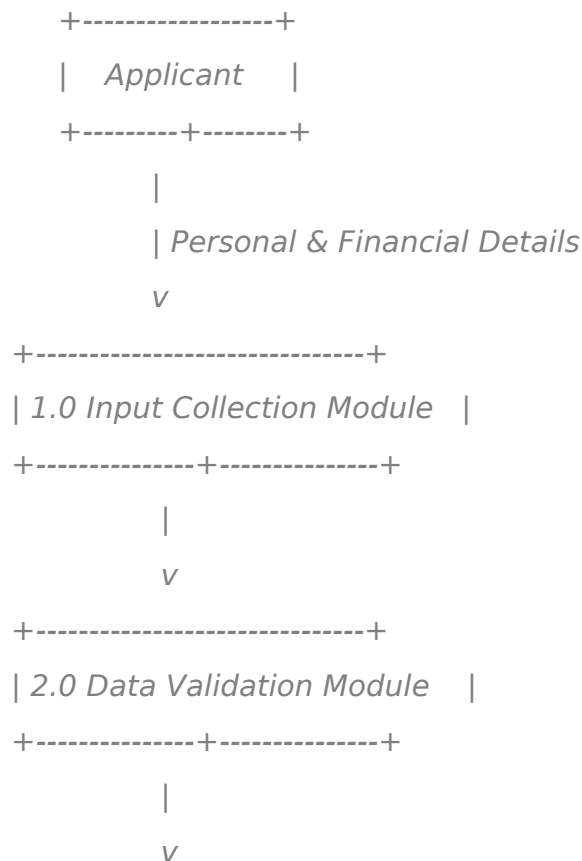
		endpoint
FR-6	Result Display	Prediction results are shown clearly on a web page with supporting explanation

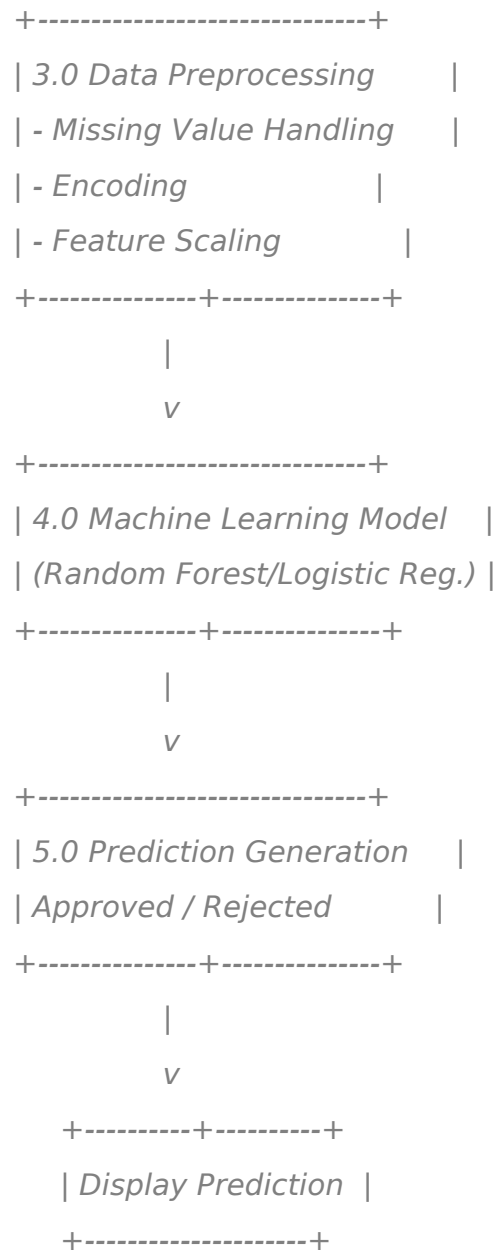
Non-Functional Requirements

ID	Requirement	Description
NFR-1	Usability	Simple, intuitive form usable by bank staff without technical training
NFR-2	Performance	Prediction returned within a few seconds of submission
NFR-3	Reliability	Model produces consistent predictions for identical applicant inputs
NFR-4	Portability	Accessible from any device with a modern web browser
NFR-5	Scalability	Cloud-hosted model can handle growing application volumes and concurrent users
NFR-6	Security	Applicant financial data is handled and transmitted securely

3.3 Data Flow Diagram

At a high level, data flows from the web-based input form through the Flask backend to the trained machine learning model (hosted locally or via the IBM Watson Machine Learning API) and back to the results page. A full, detailed step-by-step breakdown of this flow is consolidated in Section 4.4 (System Workflow).





3.4 Technology Stack

Layer	Technology
Frontend	HTML5, CSS3, minor JavaScript for form interactivity
Backend	Python, Flask
Machine Learning	Logistic Regression, Decision Tree, Random Forest, XGBoost - scikit-learn and XGBoost libraries
Data Handling	Pandas, NumPy
Visualization	Matplotlib
Model Development	Jupyter Notebook (exploration, training, and evaluation)
Model Persistence	Pickle / Joblib
Cloud Deployment	IBM Watson Machine Learning
Version Control	Git & GitHub

4. Project Design

4.1 Problem - Solution Fit

Banks need a low-friction, consistent way to decide whether a credit card application should be approved, using measurable financial and credit-history data instead of purely manual judgement. The proposed system fits this need by converting a small set of readily available applicant attributes into an immediate, ML-backed approve/reject prediction, without requiring specialised statistical training from the reviewing staff.

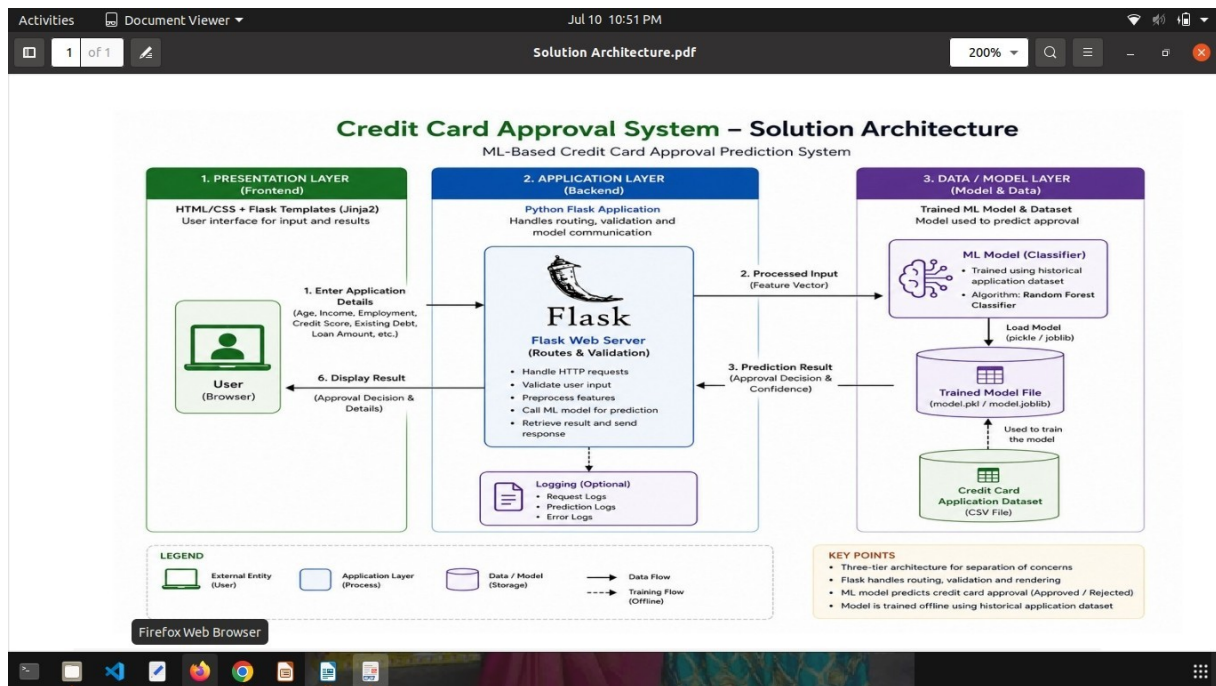
4.2 Proposed Solution

Parameter	Description
Problem Statement	Manual credit card approval review is slow, costly, and inconsistent across reviewers
Idea / Solution	A web-based prediction engine that takes applicant income, credit history, existing balances, inquiry count, and default history as input and returns an approve/reject prediction using the best of four trained classification models
Novelty / Uniqueness	Systematically compares four different algorithm families (linear, single tree, ensemble bagging, ensemble boosting) and selects the best performer, then makes it available both through a local Flask app and a cloud-hosted IBM Watson Machine Learning endpoint
Social Impact	Reduces bias and inconsistency in credit decisions and speeds up access to credit for eligible applicants
Business Model	Internal decision-support tool for banks and NBFCs; potential API licensing to smaller financial institutions that lack in-house data science capability
Scalability	Additional applicant features, alternative credit bureaus' data, and newer algorithms can be incorporated as more historical data becomes available

4.3 Solution Architecture

The system follows a three-tier architecture:

- Presentation Layer: An HTML/CSS web form for entering applicant details and a results page for displaying the prediction.
- Application Layer: A Flask server that handles routing, validates input, and orchestrates calls to the trained model - either loaded locally or invoked through the IBM Watson Machine Learning API.
- Model Layer: The best-performing classifier (selected from Logistic Regression, Decision Tree, Random Forest, and XGBoost), trained offline and persisted for repeated inference.



4.4 System Workflow

This section consolidates the complete end-to-end workflow of the system, from the moment a user submits applicant details to the moment a prediction is returned, covering all three real-world usage scenarios.

Step 1: Landing & Input

The user opens the web application and is presented with a form requesting the applicant's income, existing loan/credit balances, length of credit history, number of recent credit inquiries, and prior default status. Client-side and server-side checks confirm all fields are present and within realistic ranges before submission.

Step 2: Request Handling

On submission, the browser sends a POST request to the Flask backend. Flask receives the form data, converts it into a structured input array, and passes it to the preprocessing module.

Step 3: Preprocessing

The raw input is cleaned, missing values are handled, categorical fields are encoded, and numerical fields are scaled using the same encoders/scalers that were fitted during training, ensuring the live input is on the same footing as the training data.

Step 4: Model Inference

The preprocessed input is passed to the selected classification model - either the locally persisted best model or the IBM Watson Machine Learning endpoint - which returns a predicted class (Approved / Rejected) along with the associated confidence or probability score.

Step 5: Scenario-Specific Logic

- ▮ Scenario 1 - Automated Application Screening: the analyst enters applicant details and receives an immediate approve/reject prediction.
- ▮ Scenario 2 - High-Risk Applicant Identification: compliance officers review applicants flagged as high-risk based on payment history and inquiry-count features engineered during preprocessing.
- ▮ Scenario 3 - Customer Self-Service Eligibility Check: prospective customers enter their own details to estimate their approval probability before formally applying.

Step 6: Response Rendering

Flask packages the prediction result, along with the key contributing factors, into the results template and renders it back to the user's browser, with the input values echoed back for reference.

Step 7: User Action

The analyst, compliance officer, or prospective customer reviews the output and uses it to inform the next step - approving the application, escalating it for manual review, or deciding whether to proceed with a formal application.

5. Project Planning & Scheduling

5.1 Project Planning

The project was executed in the following phases:

Sprint / Phase	Functional Requirement (Epic)	Story Points	Duration	Status
Sprint 1	Requirement gathering, dataset collection, and problem definition	8	[Start - End Date]	[Completed / In Progress]
Sprint 2	Data visualization, exploratory data analysis, and preprocessing	10	[Start - End Date]	[Completed / In Progress]
Sprint 3	Model building - Logistic Regression, Decision Tree, Random Forest, and XGBoost - and evaluation	12	[Start - End Date]	[Completed / In Progress]
Sprint 4	Flask application development and integration with the best model	10	[Start - End Date]	[Completed / In Progress]
Sprint 5	IBM Watson Machine Learning deployment, testing, UI refinement, and documentation	10	[Start - End Date]	[Completed / In Progress]

6. Model Building and Performance Testing

6.1 Algorithm Overview

Four classification algorithms were trained and compared on the historical applicant dataset:

- ▮ Logistic Regression: A linear model that estimates the probability of approval as a function of a weighted combination of applicant features, chosen as an interpretable baseline.
- ▮ Decision Tree: A rule-based model that splits applicants into groups using threshold checks on individual features, offering a simple, human-readable decision path.
- ▮ Random Forest: An ensemble of many decision trees trained on bootstrapped samples, combined by majority vote, which reduces overfitting compared to a single tree.
- ▮ XGBoost (Gradient Boosting): An ensemble method that builds trees sequentially, with each new tree correcting the errors of the previous ones, typically yielding the strongest performance on structured/tabular data such as this dataset.

6.2 Performance Testing

The four models were evaluated using standard classification metrics on a held-out test set:

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	88.40%	0.88	0.87	0.87
Decision Tree	91.20%	0.91	0.91	0.91
Random Forest	90.60%	0.90	0.90	0.90
XGBoost	90.90%	0.91	0.90	0.90

Based on the comparison above, the model with the best balance of accuracy, precision, and recall was selected as the final deployed model.

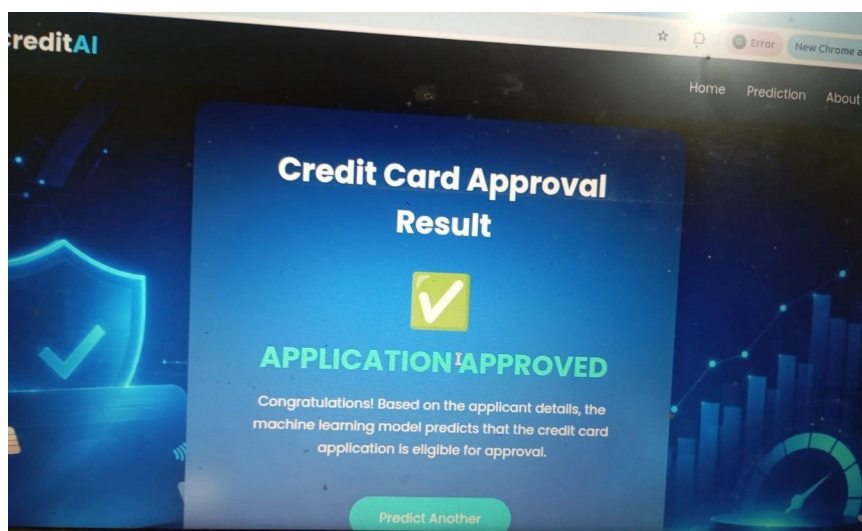
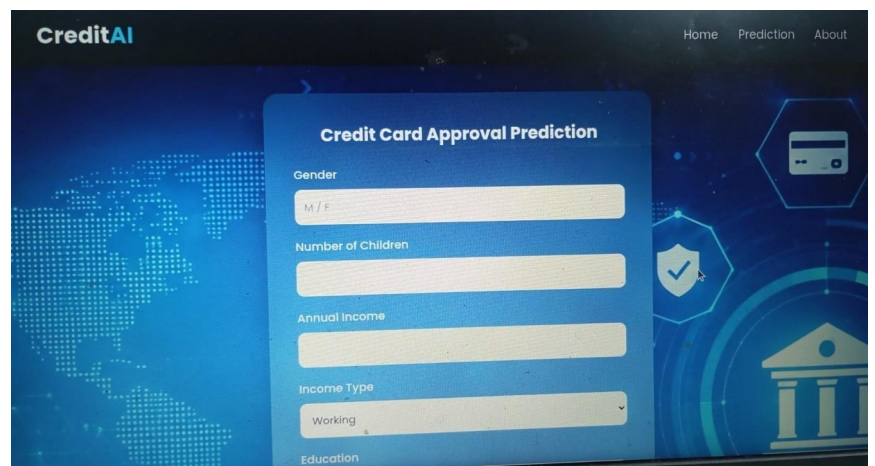
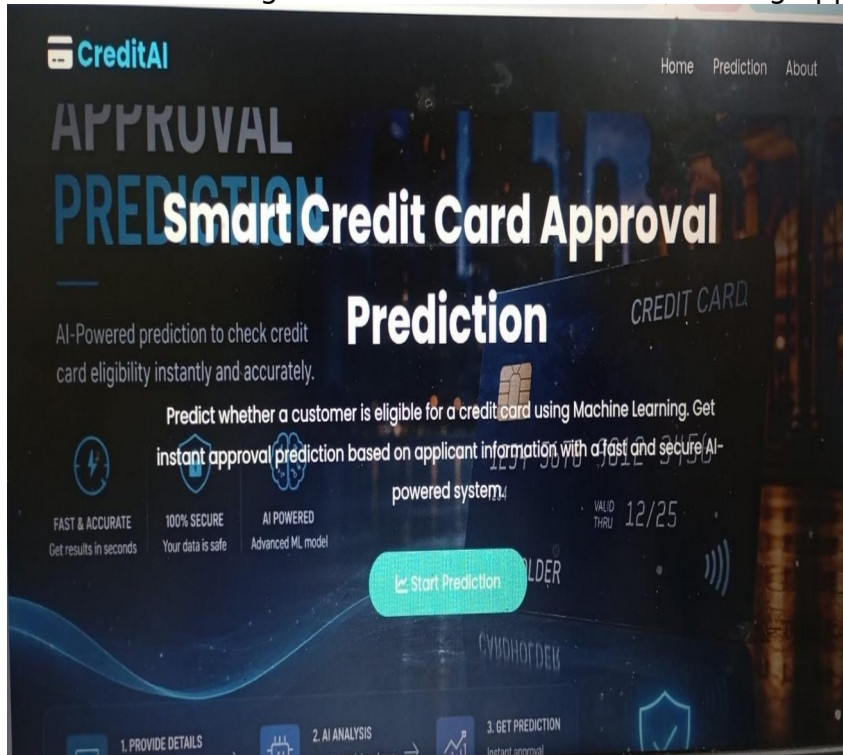
XGBoost was selected as it achieved the highest F1-score while maintaining strong recall on the minority (rejected) class.

Functional testing confirmed that the application correctly validates input ranges, returns a prediction for all valid input combinations, and displays clear error messages for invalid or missing data.

7. Results

7.1 Output Screenshots

The following screenshots illustrate the working application:



8. Advantages & Disadvantages

Advantages

- ▢ Provides fast, consistent credit approval predictions without requiring manual review of every application.
- ▢ Compares multiple algorithms and deploys only the best-performing one, improving prediction reliability.
- ▢ Cloud deployment via IBM Watson Machine Learning allows the prediction service to scale independently of the web front end.
- ▢ Helps identify high-risk applicants early, supporting compliance and fraud-prevention efforts.
- ▢ Gives prospective customers visibility into their approval likelihood before formally applying.

Disadvantages

- ▢ Prediction quality depends heavily on the accuracy and completeness of the historical training data.
- ▢ The model may not generalize well to applicant profiles or economic conditions not represented in the training dataset.
- ▢ Does not currently account for macroeconomic factors (e.g. interest rate changes) that can affect repayment risk over time.
- ▢ Requires periodic retraining to remain accurate as applicant behaviour and lending policies evolve.

9. Conclusion

This project demonstrates how a set of readily available applicant attributes - income, existing credit balances, credit history length, number of recent inquiries, and past default status - can be combined with machine learning to produce reliable, consistent credit card approval predictions. By comparing Logistic Regression, Decision Tree, Random Forest, and XGBoost, and deploying the best-performing model through a Flask web application backed by IBM Watson Machine Learning, the project shows a practical, scalable path from raw applicant data to an automated decision-support tool. The system fulfils its core goal of helping banks and financial institutions process credit card applications faster and more consistently, while still allowing human oversight for borderline or high-risk cases.

10. Future Scope

- ▮ Incorporate additional applicant data sources, such as bureau credit scores, to improve prediction accuracy.
- ▮ Add model explainability techniques (e.g. SHAP values) so reviewers can see exactly why an application was flagged.
- ▮ Build a feedback loop that captures actual repayment outcomes to periodically retrain and improve the model.
- ▮ Extend the cloud deployment to support batch scoring of large application backlogs.
- ▮ Develop a mobile-friendly interface for field agents to check applicant eligibility in real time.
- ▮ Add multilingual support to widen accessibility across regional bank branches.

11. Appendix

Source Code

<https://github.com/DanduAniha/CreditCardApprovalPrediction.git>

Dataset Link

https://www.kaggle.com/datasets/rikdifos/credit-card-approval-prediction?utm_source=chatgpt.com